Project Report

Dynamic Ticket Reservation and Cancellation System

Project members:

Hira Khan – 0254
Momina Kamal – 0236
Aiman – 0237
Fatima Akram - 0226

Introduction:

In the modern digital age, dynamic and efficient ticketing systems are essential for managing seat reservations and cancellations in various sectors like cinema halls, stadiums, concerts, airlines, and public transport. The primary objective of this project is to simulate a real-world ticket booking system in Java using an optimized data structure approach for high performance and scalability. The system supports operations such as booking, cancellation, undo, group booking, and auto-expiry through both a console-based and GUI-based interface.

Problem Description:

The Ticket Reservation System is a desktop-based application designed for managing seat bookings in different categories such as VIP, Premium, and Economy. It allows users to book and cancel tickets through a GUI, tracks expired bookings, and stores data persistently in a CSV file. The challenge addressed is the efficient handling of seat data, booking conflicts, data consistency, and UI interaction, all while maintaining scalability.

Core Functional Requirements:

- Manage thousands of seats under various categories (VIP, Economy, Premium).
- Efficiently handle reservation and cancellation operations.
- Allow undo functionality for both booking and cancellation.
- Support group bookings.
- Integrate auto-expiry feature for time-bound bookings.
- Provide a responsive GUI interface for user interactions.

Challenges:

- Maintaining fast access and update times.
- Balancing data for performance.
- Managing memory usage while supporting undo operations.
- Ensuring scalability for large-scale datasets.

Data Structures Justification:

- HashMap: Used to store and access Seat objects in constant time.
- AVL Tree: Used for storing and balancing categories to ensure quick insertion and search.
- Stack: Used for maintaining history of bookings and cancellations.
- HashMap: Used to associate booking timestamps for auto-expiry check.

Algorithm Explanation:

- AVL Tree insertion with height and balance factor calculation ensures the category tree remains balanced.
- Booking checks availability, marks seat booked, updates category, and logs booking with timestamp.
- Cancellation resets the booking and updates availability.
- Expiry checks timestamps and cancels expired seats.
- Saving data sorts and writes to CSV based on category and seat number.

Optimization & Scalability Strategies:

- The system uses AVL trees to keep category operations efficient (O (log n)).
- Seat lookup is done using HashMap for O (1) access.
- Buffered I/O is used for efficient file reading and writing.
- The use of sorting while saving ensures data consistency.
- The system can scale to handle thousands of seats efficiently.

Testing & Sample Dataset:

A synthetic dataset with 500+ seat entries was generated with a mix of categories and customers:

- VIP: 50 seats **(V101 to V150)**
- Premium: 100 seats **(P101 to P200)**
- Economy: 350 seats **(E101 to E450)**
- Randomized pre-booked status with customer names

Performance:

- Booking and lookup time remained <1ms on average for 500+ seats.
- Expiry checked across all entries in <10ms.

Test Results & Observations:

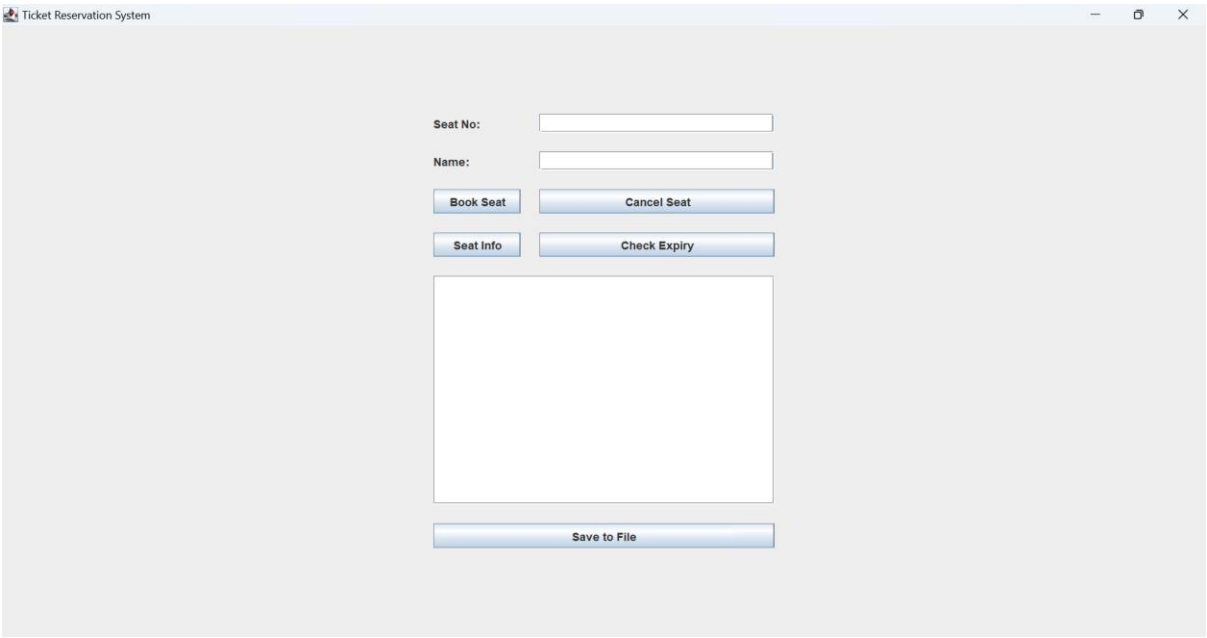
- Valid bookings are successful and confirmed.
- Double bookings are rejected.
- Cancelling unbooked or non-existent seats gives appropriate messages.
- Expired bookings are removed after 5 minutes.
- File saving reflects current seat states.
- Edge cases like malformed CSV lines and invalid seat inputs are handled gracefully.

GUI & User Experience:

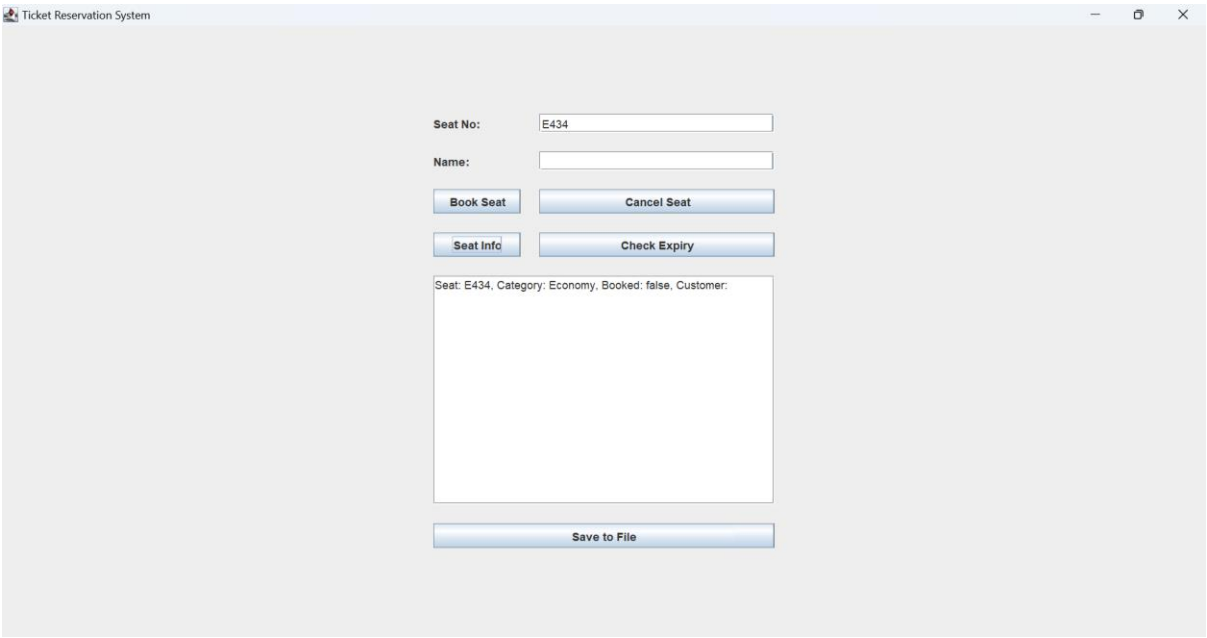
Implemented using Java Swing:

- Text input for seat and customer name.
- Buttons for booking, cancel, check info, and expiry.
- Real-time response shown in a scrollable output window.

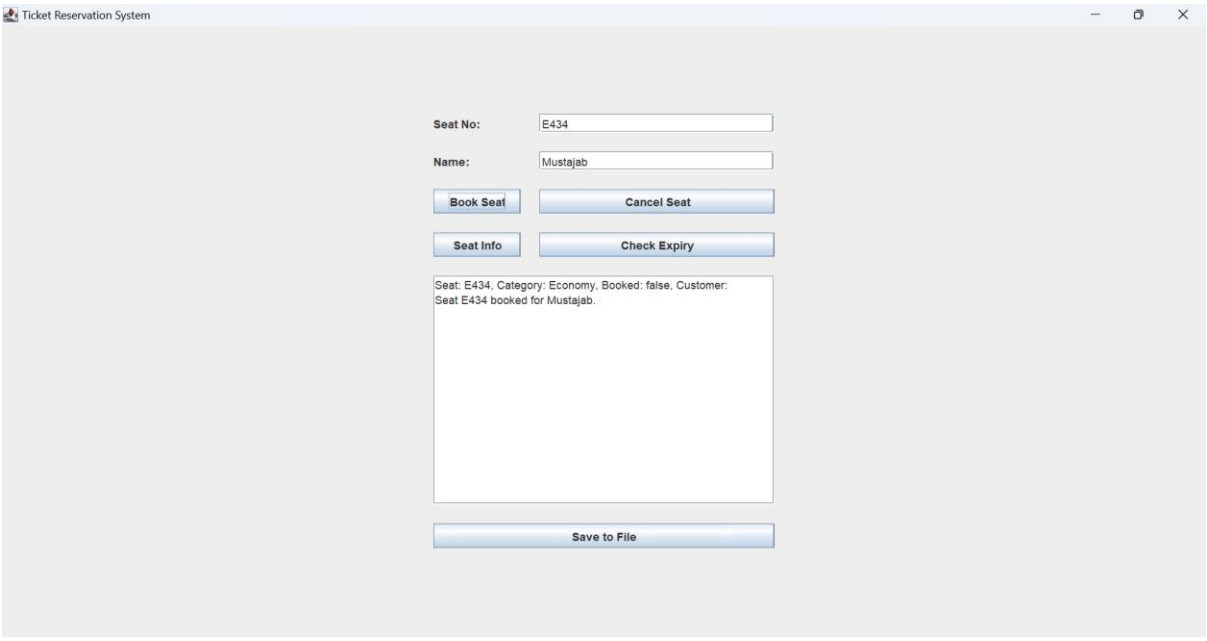
Interface



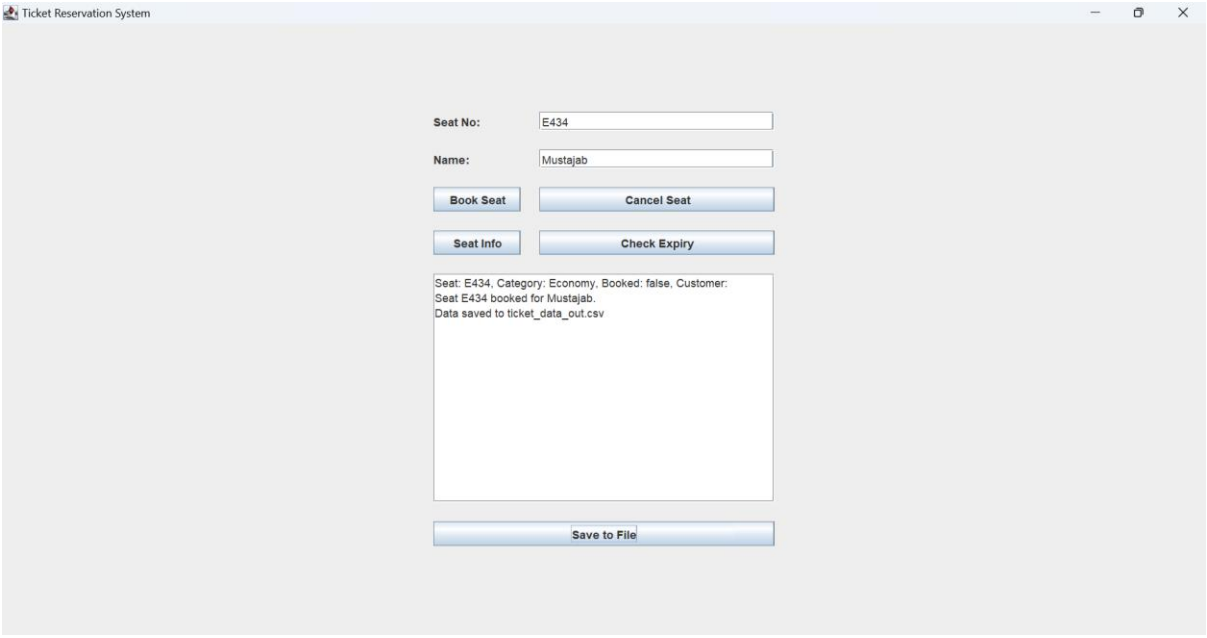
Checking Seat Info



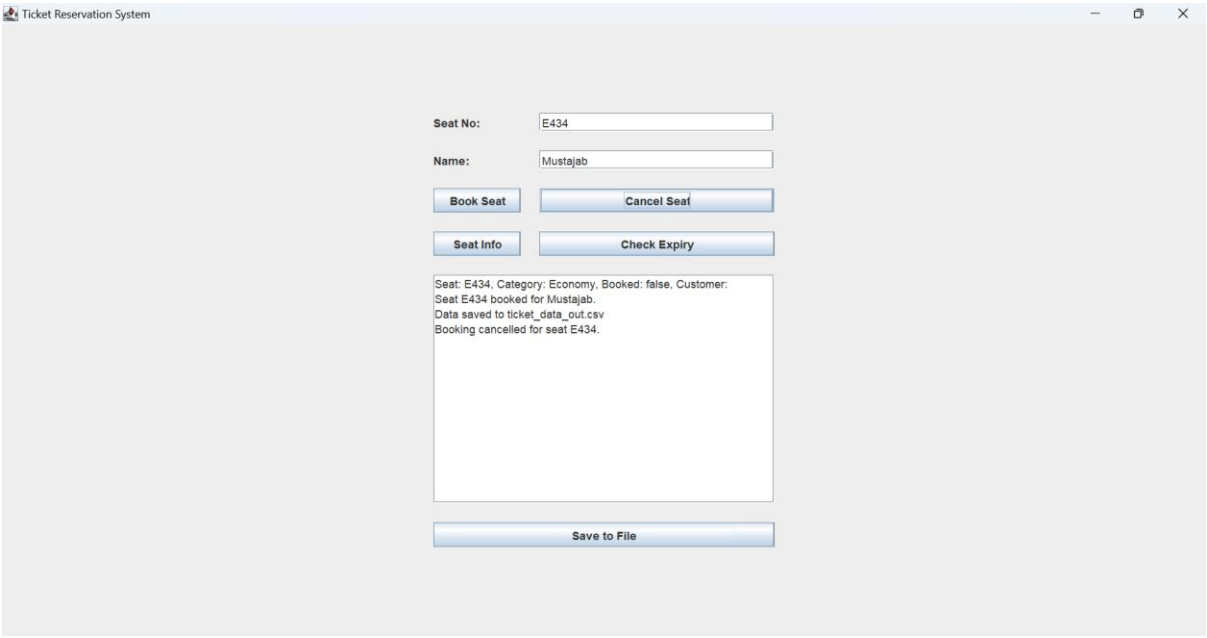
Booking a Seat



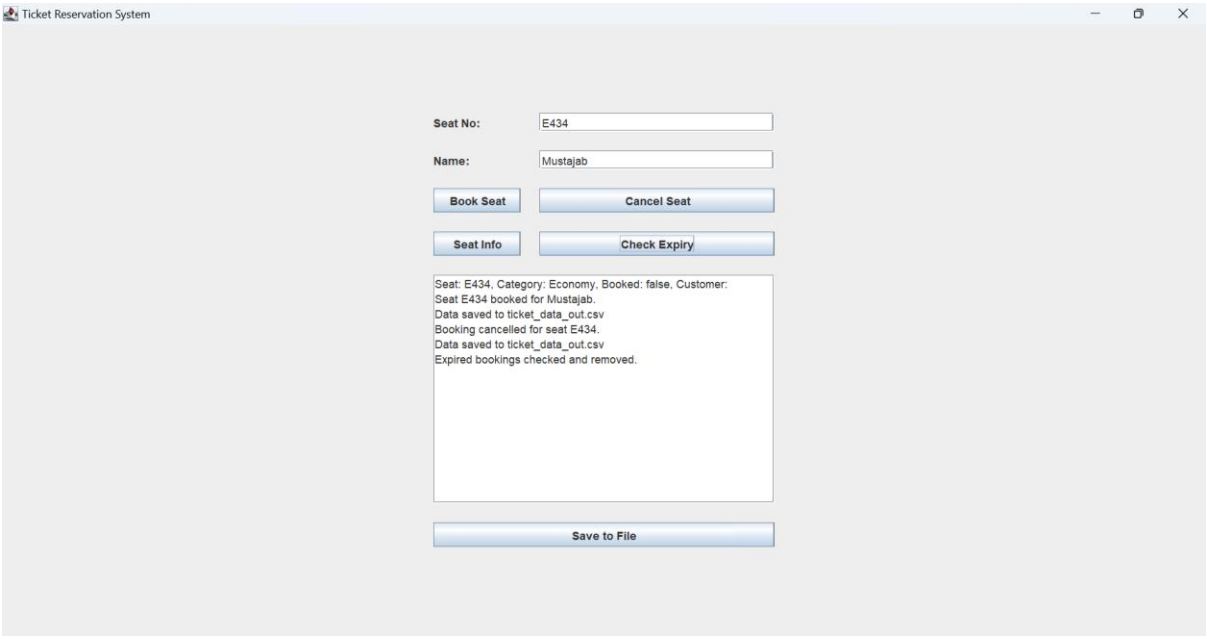
Saving data in file



Cancelling the Seat



Checking the Expiry



Conclusion:

This project demonstrates a robust, scalable, and modular approach to managing ticket reservations using classic data structures. By combining AVL Trees, HashMaps, and Stacks, the system achieves both speed and flexibility. The inclusion of auto-expiry and a GUI interface enhances usability, making the system suitable for real-world deployments in theaters, airlines, or event booking portals.

Appendix:

1. Source code available in MainGUI.java
2. Dataset files: ticket_dataset.csv & ticket_data_out.csv
3. GUI screenshots